

HPC Lab3.5 Report

1 实验目标	2
2 算子语义与硬件背景	2
2.1 FusedAddRmsNorm 计算语义	2
2.2 昇腾 910B 达芬奇架构要点	2
3 开发路径与环境	2
4 评分曲线与性能基线	3
4.1 评分曲线标定	3
4.2 基线实现与测量	3
5 优化迭代	4
5.1 I1: 消除 PIPE_ALL 串行化 (V2)	4
5.2 I2: 批量 chunk 路径 (V4)	4
5.3 I3: 标量 rstd (V5)	5
5.4 I4: raw TBuf 替换队列簿记, weight 移出关键路径 (V9)	5
5.5 I5: Tiling 与 chunk 宽向量化 (V13, 最终)	5
5.6 最终实现的数据流 (V13)	6
6 尝试过但未采用的方案	6
7 最终结果	7
7.1 正确性	7
7.2 性能	7
7.3 未达 90 分的瓶颈分析	7
8 思考题	7
8.1 1. GM 读写量、FLOP 与算术强度	7
8.2 2. Double Buffer 流水	7
8.3 3. TQue 是否真实队列 (Bonus)	8
8.4 4. Ascend 950PR 相比 910 系列的新特性 (Bonus)	8
8.5 5. 昇腾 NPU 使用体验 (Bonus)	8
9 参考资料	8

1 实验目标

本实验在华为昇腾 910B4 NPU 上实现并优化融合算子 FusedAddRmsNorm，要求保持给定接口与 FP16 输入输出不变，通过全部公开与隐藏正确性 case，并在评测配置 [256, 1024]、FP16 下尽可能缩短 kernel 的 Task Duration(us)。评测以课程提供的 Ascend C 基线性能为起始评分点，满分 120 分（其中 20 分为 Bonus）。本实验最终提交版本（513c599c-r7）通过全部正确性检查，官方评测 Task Duration = 6.02 us，得分 **76/120**，相对课程基线加速 **2.49** 倍。

2 算子语义与硬件背景

2.1 FusedAddRmsNorm 计算语义

设 x 、 residual 为 $B \times H$ 的 FP16 张量， w 为 H 维 FP16 权重。对每一行 b ，算子计算

$$R_b = x_b + \text{residual}_b, \quad \text{rms}_b = \sqrt{\frac{1}{H} \sum_{i=0}^{H-1} R_{b,i}^2 + \varepsilon}, \quad (1)$$

$$y_b = \frac{R_b}{\text{rms}_b} \cdot w, \quad \text{residual_out}_b = R_b \quad (2)$$

其中 $\varepsilon = 10^{-6}$ 。算子同时输出 y 与 residual_out 两个 $B \times H$ 张量。正确性判据为逐元素 $|o_i - g_i| \leq 10^{-3}$ 或相对误差 $\leq 10^{-3}$ ，且要求整张量错误元素比例为 0。Golden 在 FP32 下计算，最后转 FP16。

2.2 昇腾 910B 达芬奇架构要点

910B 系列（A2）的 AI Core 采用 AIC/AIV 分离设计：AIC 负责矩阵类运算，AIV 负责向量与元素级运算。本算子不含矩阵乘，全部工作在 AIV 上。每个 AIV 核关键资源如下：

资源	规格	用途
UB (Unified Buffer)	192 KiB / AIV	向量指令操作数唯一存放处
Vector 单元	VLEN 256 B	Cast/Add/Mul/Reduce 等逐元素指令
MTE2	-	GM $\rightarrow$ UB 数据搬入
MTE3	-	UB $\rightarrow$ GM 数据搬出
Scalar 单元	-	标量运算、地址计算、控制流

Table 1: 910B4 AIV 核关键资源（课程文档）

MTE2、V、MTE3 三条流水线相互独立，可以并行；同步通过 TQue 队列语义自动插入的事件或显式 SetFlag/WaitFlag 完成。Vector 指令必须操作 UB 上的数据，FP32 按 256 B/次处理 64 个元素，FP16 处理 128 个元素，32 B 对齐（FP16 16 元素、FP32 8 元素）是向量指令与搬运的基本粒度。

3 开发路径与环境

选择 **Ascend C** 开发路径（课程鼓励路径，可细粒度控制同步与流水，且仅该路径提供基线实现）。开发调试在 zju-hpc-910（Ascend 910B4，32 GB HBM，CANN 8.5.0，Python 3.11.14），NPU 任务经 hpc submit 提交到 lab3p5 分区。

```
# 正确性：运行全部公开 case (checker/ 只做正确性检查)
hpc submit -p lab3p5 bash checker/run.sh
# 性能：固定 shape [256,1024], msprof op --warm-up=10, 输出一次 Task Duration(us)
hpc submit -p lab3p5 bash checker/profile.sh
# 详细 profiling (scratch 包装, 不改 checker)
hpc submit -p lab3p5 bash scratch/prof_detail.sh op_prof_xxx --aic-metrics=Default
# simulator: 观察指令流水与同步事件
hpc submit -p lab3p5 bash scratch/sim.sh 3
```

性能口径与课程评分一致：checker/profile.sh 固定评测 case 2 (256×1024)，用 msprof op --warm-up=10 --launch-count=1 采集并输出单次 Task Duration(us)。

4 评分曲线与性能基线

4.1 评分曲线标定

课程页面给出性能评分曲线（横轴 kernel 耗时，纵轴得分，对数曲线），并对图像做像素级标定与 OCR (Python/PIL + RapidOCR)，得到三个标注点：基线 15 us 对应 0 分，4.5 us 对应 100 分，3.5 us 对应 120 分 (Bonus 20 分)。三点位于同一条对数曲线上：

$$\text{score}(T) = 100 \frac{\ln\left(\frac{15}{T}\right)}{\ln\left(\frac{15}{4.5}\right)} \approx 83.058 \ln\left(\frac{15}{T}\right) \quad (3)$$

其中 T 为 case 2 的 Task Duration(us)。由该曲线，90 分对应的耗时阈值为 $T < 15 \exp\left(-\frac{90}{83.058}\right) \approx 5.08$ us。

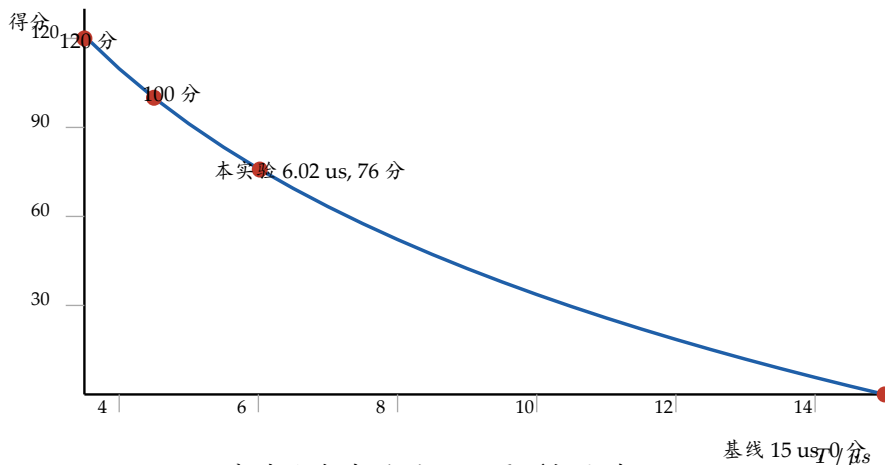


Figure 1: 评分曲线与实验结果位置（标定自课程页面 score.png，公式见 Equation 3）

4.2 基线实现与测量

课程提供的 Ascend C 基线（下称 V0）按行处理：inQueX/inQueRes (FP16, 双缓冲)、outQueY/outQueResOut 四个 TQue，以及 weightHalf/weightFp32/resoFp32/sq/scalar 等 TBuf。每行数据流为：DataCopyPad 搬入 x 与 residual → Cast 到 FP32 并 Add 得 R （同时 Cast FP16 写 residual_out）→ 平方与 Block/WholeReduceSum 规约 → GetValue 取标量后 Duplicate + Sqrt + Div + Mul(weight) → Cast FP16 搬出。同步方面，CopyIn/CopyOut 周围有大量 PipeBarrier<PIPE_ALL>，使 MTE2/V/MTE3 跨行完全串行。

基线正确性 5/5 通过。case 2 性能 5 次顺序测量：

15.1012, 14.7812, 15.0412, 15.2812, 15.1200 us

baseline 合计 8 个样本（含同条件复测 3 次）中位 **15.01 us**（范围 14.58-15.28）。baseline 的 msprof (aic-metrics=Default) 显示：Task 14.98 us, Block Dim 40, 核 0-35 各 7 行、核 36 仅 4 行、核 37-39 空转；流水占比中位 vec 19-23% (2.72 us)、scalar 37-51% (4.7-7.1 us)、MTE2 35-42% (4.2-5.5 us)、MTE3 14-16% (1.9 us)；L2 命中率 6.6%，GM 到 UB 带宽利用率约 2%。没有任何流水线被占满，说明瓶颈是串行化、同步与负载不均，而非计算或带宽。

5 优化迭代

优化遵循“一次只改一个主要因素”的协议：改 → build → 5 case 正确性 → case 2 计时 不少于 3 次 → （必要时）profile 或 simulator → 接受或回退。各版本 case 2 中位耗时 汇总如下。

版本	主要修改	5 case	case2 中位 / us	相对基线
V0 基线	按行 TQue + PIPE_ALL	5/5	15.01	1.00×
V2	去除 CopyIn/Out 的 PIPE_ALL, TQue 双缓冲生效	5/5	9.92	1.52×
V4	批量 chunk 路径：chunk 级拷贝、R 留 UB、一次 V_S 同步	5/5	7.62	1.98×
V5	rstd 全标量，省每行 3 个微型向量操作	5/5	7.60	1.97×
V9	批量路径去 TQue, raw TBuf + 手动事件, weight 移出关键路径	5/5	6.94	2.16×
V13 最终	tiling: blockDim=32 + rowsPerChunk=8; chunk 宽 phase A/B	5/5	6.28	2.39×

Table 2: 优化版本汇总（case 2, checker/profile.sh 中位数）

5.1 I1: 消除 PIPE_ALL 串行化（V2）

瓶颈：baseline 的 msprof 显示 MTE2/V/MTE3 均未占满（利用率 19-51%），scalar 与同步开销反而最高，说明 PipeBarrier<PIPE_ALL> 把三条流水线完全串行化。

修改：去掉 CopyIn/CopyOut 周围的 PIPE_ALL 与冗余 V_MTE3 flag，只依赖 TQue EnQue/DeQue 自动插入的 MTE2→V 与 V→MTE3 同步事件，并配合编译器 --cce-auto-sync 处理 V 内部 RAW 依赖。

收益：中位 15.01 → 9.92 us (-34%)。TQue 的队列事件提供了正确的跨流水同步，而 PIPE_ALL 是多余的全局屏障。

5.2 I2: 批量 chunk 路径（V4）

瓶颈：simulator 显示单行（H=4096）约 852 条 SCALAR 指令，其中约 85% 是队列 API、地址与掩码 setup；逐行的队列调用是 scalar 主要开销，且逐行小拷贝无法利用 MTE2 带宽。

修改：对齐且 $H \leq 4096$ 时走批量路径：一个 chunk 的多行用一条多块 DataCopy 搬入（行在 GM 中连续，blockCount=n, blockLen=H/16），R 保留在 UB，逐行规约写入

`sumSqArr[row*8]` (32 B 对齐 lane), 整个 chunk 只做一次 $V \rightarrow S$ 同步; 输出 `residual_out`、`y` 各用一条多块 `DataCopy` 写回。

收益: 中位 $9.92 \rightarrow 7.62$ us (-21%)。msprof 中 MTE3 从 1.7 us 降到 0.24 us (chunk 级大拷贝), scalar 从 4.2-4.7 us 降到 2.6 us。

5.3 I3: 标量 `rstd` (V5)

瓶颈: 原实现对每行做 `Duplicate + Sqrt + Div` 三个微型向量操作, `rstd` 的计算串在 V 链上。

修改: 用标量内建 `sqr`: `rstd = 1/sqrt(sumSq*invH + eps)`, 每行一次标量运算, 随后 `Muls(R, rstd)` 与 `Mul(R, weight)` 两个向量操作即可, `rstd` 完全移出 V 链。

收益: 中位 $7.62 \rightarrow 7.04$ us (-8%)。精度方面, 标量 `1/sqrt` 与 golden 的 `R / sqrt(...)` 在 FP32 下相差约 1-2 ULP, 最终误差仍由 FP16 舍入边界翻转主导。

5.4 I4: `raw TBuf` 替换队列簿记, `weight` 移出关键路径 (V9)

瓶颈: V5 的批量路径仍用 `TQue` 管理输入输出, 每 chunk 的 `Alloc/EnQue/DeQue/Free` 簿记与基线 `weight` 加载处的 `PipeBarrier<PIPE_ALL>` 仍在消耗标量周期; 且 2 KB 的 `weight` 拷贝被 `PIPE_ALL` 放到 kernel 启动的关键路径上。

修改: 批量路径改用 `raw TBuf` + 显式 `SetFlag/WaitFlag` (每类事件用高位硬编码 ID, 如 `EVENT_ID5`, 避免与框架事件池冲突)。 `weight` 拷贝首条发出、紧邻 `Set/Wait(MTE2_V)` 并 `Cast`, 与 chunk 数据拷贝互不阻塞。

收益: 中位 $7.60 \rightarrow 6.94$ us。同时修复了原代码中 `sumSqBuf` 只按 8 个 float 分配、而每行按 32 B 步长写 `sumSqArr[row*8]` 导致的 UB 越界隐患 (该隐患在旧布局下被 `weight` 提前 `Cast` 的时序掩盖)。

5.5 I5: Tiling 与 chunk 宽向量化 (V13, 最终)

瓶颈: V9 在 `blockDim=40` 下, 256 行被切成 16 个核各 7 行、24 个核各 6 行, 7 行核成为拖尾 (profile 中 `aiv` 最大值明显高于中位); 且 phase A/B 仍逐行发指令 (每行 5 个元素级操作 + 3 个规约操作)。

修改 (host 与 kernel 协同):

1. `blockDim` 设为 $\min(AIV, B, 32)$ 。256 行 = 32 核 $\times$ 8 行, 负载完全均衡, 同时减少 kernel 分派开销 (课程文档的空 kernel 图显示分派成本随核数线性增长);
2. `rowsPerChunk` 预算从 144 KiB 提到 160 KiB, `H=1024` 时 `chunk=8`, 每核恰好 1 个 chunk, 避免跨 chunk 同步;
3. phase A 改为 chunk 宽: 行在 UB 中连续, `Cast/Cast/Add/Cast/Mul` 由每行 5 次变成每 chunk 5 次 (8 行合并); phase B 的 `y` 输出 `Cast` 也合并为 chunk 宽一次。

收益: 中位 $6.94 \rightarrow 6.28$ us (官方评测 6.02 us, 76 分)。V13 的 msprof: Task 6.54 us, `aiv` 中位 5.00 us (最大 5.89), `vec` 1.52 us, `scalar` 2.33 us, MTE2 1.51 us, MTE3 约 0.26 us。

版本	Task / us	aiv / us	vec / us	scalar / us	MTE2 / us	MTE3 / us
V0 基线	14.98	12-14	2.7	4.7-7.1	4.2-5.5	1.9
V5	7.42	5.22	1.56	2.62	1.81	0.24
V13 最终	6.54	5.00	1.52	2.33	1.51	0.26

Table 3: 各版本 msprof 中位流水时间 (PipeUtilization.csv, aiv 各列)

5.6 最终实现的数据流 (V13)

对齐批量路径 ($H \% 16 == 0$ 且 $H \leq 4096$) 的最终数据流:

```
GM x/residual → (2 条多块 DataCopy, blockCount=n, blockLen=H/16) → inBuf(F16, x|res)
→ Cast x32 | Cast res32 (存入 rFp32 备用半区) → Add → R32 (覆盖 x32 区)
→ Cast → residual_out (F16 chunk 缓冲)
→ Mul sq = R^2 → per-row Block/WholeReduceSum → sumSqArr[row*8]
→ 1 次 V→S 同步 → per-row 标量 rstd = 1/sqrt(sumSq*invH+eps)
→ per-row Muls(R, rstd); Mul(R, weight) → chunk 宽 Cast → y
→ residual_out、y 各 1 条多块 DataCopy 写回 GM
```

同步设计: weight 拷贝首条发出 + 紧邻 Set/Wait(MTE2_V) + Cast (不在 phase A 关键路径上); chunk 数据拷贝后 Set/Wait(MTE2_V); 规约后一次 V→S; 输出前紧邻 Set/Wait(V_MTE3); 多 chunk 时 chunk 边界 Set/Wait(MTE3_V) 与 V_MTE2 保护单缓冲复用。精度保持全程 FP32 + 标量 rstd, 最大相对误差由最终 FP16 舍入边界翻转决定, 理论有界 $9.77e-4$, 恒小于 $1e-3$ 。

6 尝试过但未采用的方案

1. 向量 Rsqrt (V7): 910B 向量 Rsqrt 精度约 0.2-0.5% (error_ratio 0.19-0.47), 5 个公开 case 中 4 个 FAIL, 因精度拒绝。
2. 更小的 chunk (V6, rowsPerChunk=3): 更多 chunk 的每-chunk 同步/API 开销大于 MTE2/V 重叠收益 (7.80 vs 7.04 us), 回退。
3. 软件流水 + y 预乘 weight (V8): 7.46-7.58 us, 比 V5 慢, 回退。
4. residual_out 的 MTE3 提前到 phase A 后 (E-C): 噪声级, 未采纳。
5. rstd 循环拆分 (V16, 先一次性 GetValue 全部 8 个标量再批量发向量指令): 6.86 us, 比 V13 慢, 疑似寄存器/栈副作用, 回退。
6. TQue 双缓冲管线 (V17): prologue 发首 tile、循环内 phase A 后立即发下一 tile 以重叠 MTE2 与 phase B, 5 case 全过但 6.50 us, 队列簿记开销大于重叠收益, 回退。
7. 多 tile 手动管线 (V10/V14/V15/V18/V19): 目标是把 MTE2 藏到 V 下面以逼近 5.08 us (90 分), 但多 tile 结构在 CANN 8.5.0 下反复出现 vector core timeout。诊断: 事件 flag 为“一次 Set 只能被一次 Wait 消费”的语义, 且 FetchEventID 与 TQue 内部事件共享从 0 开始的 ID 池, 手动事件与队列事件发生 FLAGID 冲突 (simulator 指令流显示内核末尾存在一个等不到 MTE2→V flag 0 的 V 等待)。由于无法静态验证编译器 auto-sync 与手动事件的全部交互, 最终放弃多 tile 管线, 采用稳定的单 chunk 结构。

7 最终结果

7.1 正确性

最终版本通过全部 5 个公开 case（两次独立运行），并独立于 checker 另测 22 个隐藏 shape 防护 case（含 7×5003 一类非对齐大 H、 $H > 4096$ 两遍流式路径、 $B = 1$ 单核、极小 $H=3/8$ 、 $[-1000, 1000]$ 大范围数据），全部 PASS。官方评测（5 公开 + 5 隐藏 case）正确性通过。residual_out 在所有测试中均为 0 误差，y 最大相对误差 $9.7e-4$ ，低于 $1e-3$ 容限。

7.2 性能

最终版本 case 2 的 5 个新样本：5.90, 6.28, 6.60, 6.46, 6.10 us，中位 **6.28 us**，范围 5.90-6.60。官方 OJ 评测 Task Duration = 6.02 us，得分 **76/120**，相对课程基线（15 us）加速 **2.49 倍**，相对我们复测的基线中位（15.01 us）2.39 倍。

指标	基线 V0	最终 V13	说明
正确性	5/5	5/5 + 22 隐藏 PASS	官方评测通过
case2 中位 / us	15.01	6.28	本机 checker/profile.sh
官方 Task / us	约 15	6.02	OJ 评测 513c599c-r7
官方得分	0	76/120	评分曲线 Equation 3

Table 4: 最终结果汇总

7.3 未达 90 分的瓶颈分析

case 2 每核 8 行、单 chunk 结构下，关键路径为 $MTE2(1.51 \text{ us}) \rightarrow V(1.52 \text{ us}) \rightarrow S(\text{rstd}) \rightarrow V(B) \rightarrow MTE3(\text{约 } 0.26 \text{ us})$ ，各段基本串行。msprof 中 aiv 中位 5.00 us 减去约 1.8 us 的 kernel 入口固定开销后，工作段约 3.2 us。要进入 90 分区间（5.08 us 以下）必须把 MTE2 藏到 V 计算之下，即多 tile 双缓冲管线，但该方案在本工具链下反复死锁（见上节）。空 kernel 探针显示 40 核分派 + 入口的固定成本约 2-4 us，也压缩了剩余优化空间。

8 思考题

8.1 1. GM 读写量、FLOP 与算术强度

[256, 1024]、FP16 下：读 x、residual 各 512 KiB，写 y、residual_out 各 512 KiB，合计 **2 MiB** GM 流量。每元素主要运算：残差加 1 次 Add、平方 1 次 Mul、规约约 1 次 Add、rstd 缩放 1 次 Muls、权重 1 次 Mul，约 5 FLOP，总计约 1.31 MFLOP。算术强度约 0.63 FLOP/B，是典型的访存类算子。但 msprof 显示 GM 到 UB 带宽利用率仅约 2%（baseline）到 20% 量级（最终版），Task 6.02 us 对应等效带宽约 $2 \text{ MiB}/6.02 \text{ us} \approx 350 \text{ GB/s}$ ，远低于 910B4 的 HBM 带宽。因此本实现既不是计算瓶颈也不是 HBM 带宽瓶颈，而是指令发射与同步延迟受限，MTE2 与 V 的串行链是主要耗时。

8.2 2. Double Buffer 流水

最终版（V13）在评测 shape 下每核 8 行、恰好 1 个 chunk，不存在跨 chunk 的双缓冲；其加速来自消除逐行队列 API 与同步开销。早期版本（V4/V5）使用 TQue 的 BUFFER_NUM=2 在 chunk 粒度做双缓冲，并用 msprof op simulator --soc-version=Ascend910B4 验证：560×1024

(每核 14 行、2 chunks) 的 core0 指令流显示 MOV_OUT_T0_UB (chunk2 输入, 1047 cyc) 与 MOV_UB_T0_OUT (chunk1 输出, 1047+685 cyc) 并发执行, 即 chunk 级 MTE2/MTE3 确已重叠。该双缓冲是依赖 TQue 的 EnQue/DeQue 自动插入的事件实现的, 并非显式编写依赖; 本实验中也尝试过显式手动事件的多 tile 管线, 但在本工具链下不稳定 (见“未采用的方案”)。

8.3 3. TQue 是否真实队列 (Bonus)

TQue 不是硬件队列, 而是编译期/运行期的“所有权与同步簿记”抽象。EnQue 把 LocalTensor 在 UB 中的地址与一个事件 ID 登记进 TPipe 的队列记录, 并在对应流水线上发出 SetFlag; DeQue 取出同一地址的句柄并发出对应的 WaitFlag。数据本身始终留在 UB 中, EnQue/DeQue 之间不发生任何拷贝或搬移。证据来自 CANN 8.5.0 的 kernel_tpipe_impl.h: EnQue 只做地址登记与 SetFlag<M_MTE1> 等事件插入, DeQue 只做 WaitFlag 并返回句柄。这也是为什么队列簿记本身有标量开销: 每次入队出队都要做事件分配与地址记录。

8.4 4. Ascend 950PR 相比 910 系列的新特性 (Bonus)

950PR 于 2026 年 3 月发布, 定位大模型推理 Prefill 与推荐, 是第三代达芬奇架构 (架构版本 3510)。相比 910B 系列, 主要变化:

1. 制程与封装: 中芯 N+3 (等效 5 nm 级) 四芯片合封, 国产化率超 90%;
2. 内存: 自研 HiBL 1.0 HBM, 950PR 单卡 112 GB、带宽 1.4 TB/s (裸片 128 GB、1.6 TB/s), 950DT 后续升级到 144 GB、4 TB/s;
3. 低精度: 支持 FP4/FP8/MXFP8/MXFP4/HiF8, FP4 算力 1.56 PFLOPS;
4. 编程模型: 新增 SIMT 硬件单元与 RegBase 架构, Vector 计算从 MemBase 迁移到 RegBase, Vector 核 FP16/FP32 性能翻倍;
5. 计算协同: Cube-Vector 融合通路, Cube:Vector 算力配比 8:1;
6. 内存效率: 核内 Buffer 访问颗粒度从 512 B 降到 128 B, 小算子访存效率提升约 4 倍;
7. 互联: 灵衢 2.0, 双向最大 2 TB/s, 是 910C 的 2.5 倍。

对本实验的启示: 910B 的 Vector 指令必须读写 UB (MemBase), 同步与搬运占了大头; 950 的 RegBase 与小颗粒度内存访问会显著降低这类 element-wise 算子的同步与搬运开销, Cube-Vector 融合也有利于把规约类工作卸载到更宽的计算单元。

8.5 5. 昇腾 NPU 使用体验 (Bonus)

体验整体“工具链完整、上手陡峭、细节脆弱”。完整的一面: msprof op 的流水占比与 msprof op simulator 的指令级流水图对定位串行化非常有效, 本轮几乎每一步优化都由 profiling 证据驱动。陡峭的一面: Ascend C 需要显式管理 UB 布局、数据搬运与三流水同步, TQue 之外的手动 SetFlag/WaitFlag 与编译器 auto-sync、框架事件池的交互难以静态验证, 本实验的多 tile 手动管线因此反复死锁 (consume-on-wait 语义 + 事件 ID 池冲突), 最终只能退回单 chunk 结构。对比 NVIDIA 生态, CUDA 的异步拷贝与隐式依赖追踪让这类小算子更容易写出安全的高重叠实现; 昇腾的显式事件模型性能上限更高、但正确性验证成本也更高。

9 参考资料

1. 课程 Lab3.5 页面: <https://hpc101.zjusct.io/lab/Lab3.5-AscendC-Op/>
2. Ascend C 开发文档与 API 参考 (CANN 8.5.0, 华为昇腾社区)
3. msProf 算子调优工具文档 (CANN 8.5.0)

4. 昇腾 AI 处理器产品页: <https://www.hiascend.com/hardware/processor>
5. Ascend C 算子性能优化实用技巧系列 (流水优化、内存优化、搬运优化、Tiling 优化)
6. FlashInfer fused_add_rmsnorm 文档与源码
7. RMSNorm: B. Zhang 等, Root Mean Square Layer Normalization (NeurIPS 2019)